ITI · AI Agents DEVELOPMENT USING PYTHON TRACK

AI-Assisted Development & API Integration

Eng. Fatma Elraey

Lecture 2 — Connecting to the Brain (LLM API Integration)

Field	Detail
Instructor	Fatma Elraey
Course	AI-Assisted Development & API Integration (2-Day Course)
Session	Day 2 of 2 — Lecture, 3 Hours
Focus	Moving from using AI tools to building with AI directly — making real API calls, managing conversation state, and getting clean, structured data back instead of unpredictable text.

Quick Recap — Day 1

Yesterday you used AI assistants inside your editor — Cursor and Copilot, writing code alongside you. Today the AI moves from being a **tool you use into being a service you build with**: you'll write the actual code that talks to a language model's API, the same underlying technology that powers every AI product you've used so far. We'll use Python for the examples, but every concept here applies just as directly if you're calling these APIs from JavaScript or any other language.

Today's Outline

1. The GenAI API Landscape
2. Core Concepts of API Communication
3. Structured Outputs & Function Calling
4. Awareness Briefing: What's Next

01 The GenAI API Landscape

Every major AI chat product you've used is a polished interface sitting on top of an API — the same kind of API you're about to call directly yourself. Three providers dominate the landscape today, and while their exact syntax differs slightly, the underlying ideas are nearly identical across all three.

Figure 2.1 — The three major providers, each offering both text and vision models.

Within each provider, models are generally split by what kind of input they accept. A text model reads and writes text only. A vision-capable model can also accept an image as part of the input — useful for tasks like describing a photo, reading text from a screenshot, or checking a UI design against a mockup.

```
# example — text model vs vision-capable model
TEXT MODEL  Input: "Summarize this paragraph."
VISION-CAPABLE MODEL  Input: screenshot + "Why is this UI button misaligned?"
Key idea: vision adds image understanding; it still can work with text too.
```

To call any of these APIs, you'll install the provider's official Python package and authenticate with an API key. Here's the shape for OpenAI, which we'll use for the rest of today's examples (Anthropic and **Google's SDKs** follow the same basic pattern):

```
pip install openai --break-system-packages


from openai import OpenAI

client = OpenAI()  # reads OPENAI_API_KEY from the environment


# example — API key: keep secrets out of source code
GOOD  Store OPENAI_API_KEY in the environment.
Python  client = OpenAI()  # SDK reads the environment variable
AVOID  api_key = "sk-..." inside a .py file or Git repository.
Why?  Source code may be shared; secrets should not be committed.
```

PRO TIP *It genuinely doesn't matter much which provider you start with — the messages/roles pattern, the way you manage conversation history, and the way you request structured output are nearly identical across OpenAI, Anthropic, and Google.*

✓ QUICK CHECK — The GenAI API Landscape

Q1. Name the three major GenAI API providers covered today.

Answer: *OpenAI, Anthropic, and Google.*

Q2. What's the difference between a text model and a vision-capable model?

Answer: *A text model only reads and writes text; a vision-capable model can also accept an image as part of the input.*

02 Core Concepts of API Communication

Every message you send to a chat model has a role attached to it. There are three roles you'll use constantly: system sets the model's overall behavior and personality for the whole conversation; user is what the human is asking; assistant is what the model has said back. You send all of them together, every single time, as a list:

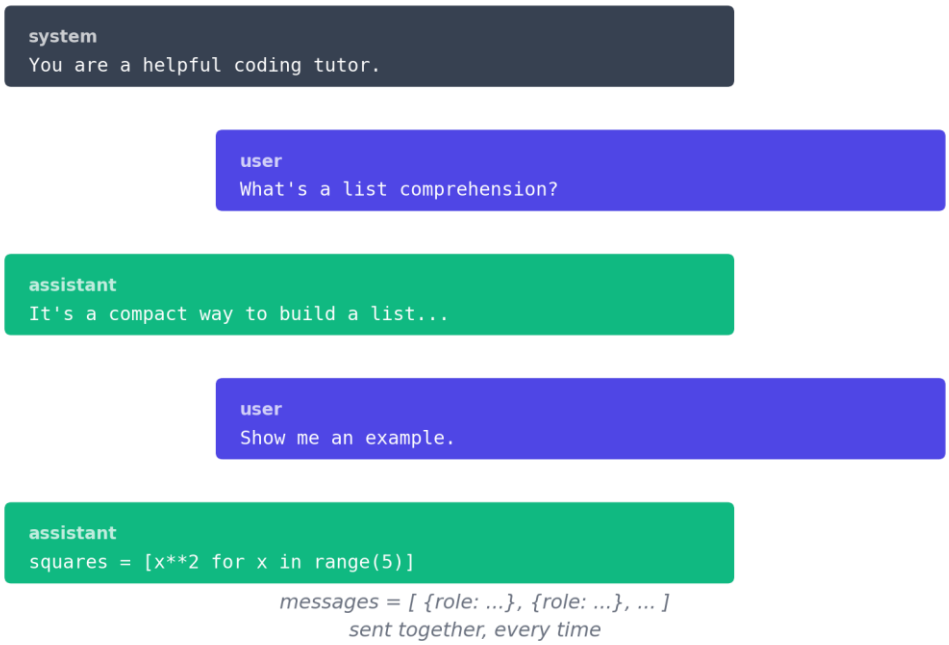

Figure 2.2 — A conversation is just a growing list of role-tagged messages.

```
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "You are a helpful coding tutor."},
        {"role": "user", "content": "Explain what a list comprehension is."}
    ]
)
print(response.choices[0].message.content)


# example — the three message roles
system     "You are a patient Python tutor. Use simple examples."
user       "Explain dictionaries."
assistant  "A dictionary stores key-value pairs..."
Think of roles as labels that tell the model who said what.
```

Or you can use **Gemini** free trial:

```python
from dotenv import load_dotenv
from google import genai
from google.genai import types
import os

load_dotenv()

client = genai.Client(
    api_key=os.getenv("GEMINI_API_KEY")
)

# Create a chat session with your system instruction
chat = client.chats.create(
    model="gemini-3.6-flash",
    config=types.GenerateContentConfig(
        system_instruction="You are a helpful coding tutor."
    )
)

# Send your message through the chat object
response = chat.send_message("Explain what a list comprehension is.")

print(response.text)
```

Here's the part that surprises most beginners: the API has **no memory** of its own. Every request is completely **independent** — if you want the model to "remember" earlier turns of a conversation, you have to resend the entire conversation history yourself, every single time, with the new message appended to the end.

```
messages = [
    {"role": "system", "content": "You are a helpful assistant."}
]

def chat(user_input):
    messages.append({"role": "user", "content": user_input})
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=messages
    )
    reply = response.choices[0].message.content
    messages.append({"role": "assistant", "content": reply})
    return reply

print(chat("What is 12 * 8?"))
# 96
print(chat("Now divide that by 4."))
# 24  -- the model can only get this right because
#         the full history was resent


# example — why conversation history must be resent
TURN 1  User: "My favorite language is Python."
TURN 2  User: "What language did I say I like?"
WITHOUT HISTORY  The API cannot know what "I said" refers to.
WITH HISTORY     Earlier user/assistant messages are included again, so it can answer "Python."
```

Or you can use Gemini:

```
from dotenv import load_dotenv
from google import genai
from google.genai import types
import os

load_dotenv()

client = genai.Client(
    api_key=os.getenv("GEMINI_API_KEY")
)


# Initialize a chat session with system instructions
chat_session = client.chats.create(
    model="gemini-3.6-flash",
    config=types.GenerateContentConfig(
        system_instruction="You are a helpful assistant."
    )
)

def chat(user_input):
    # Send message to the session; history is managed automatically
```

```
    response = chat_session.send_message(user_input)
    return response.text

print(chat("What is 12 * 8?"))
# 96

print(chat("Now divide that by 4."))
# 24 -- the model maintains context across messages
```

Notice "Now divide that by 4" only makes sense because the previous question and answer were included in the messages list. This is also why long conversations get more expensive and eventually hit a length limit — you're resending the whole thing every time.

> **PRO TIP** *Managing **state** is entirely your responsibility as the developer. Every "AI chat app" you've used is doing exactly this messages.append() pattern behind the scenes.*

✓ QUICK CHECK — Core Concepts of API Communication

Q1. What are the three core message roles?

Answer: *system (overall behavior), user (what the human asks), and assistant (what the model has said).*

Q2. Why do you need to resend the entire conversation history on every request?

Answer: *The API has no memory of its own — each request is independent, so "remembering" earlier turns means you resend the full messages list yourself each time.*

03 Structured Outputs & Function Calling

By **default**, a model replies with free-form **text** — which is great for a chat interface, and genuinely annoying when your code needs to parse the result. Markdown formatting, extra commentary, and small wording changes between requests make plain text painful to work with programmatically.

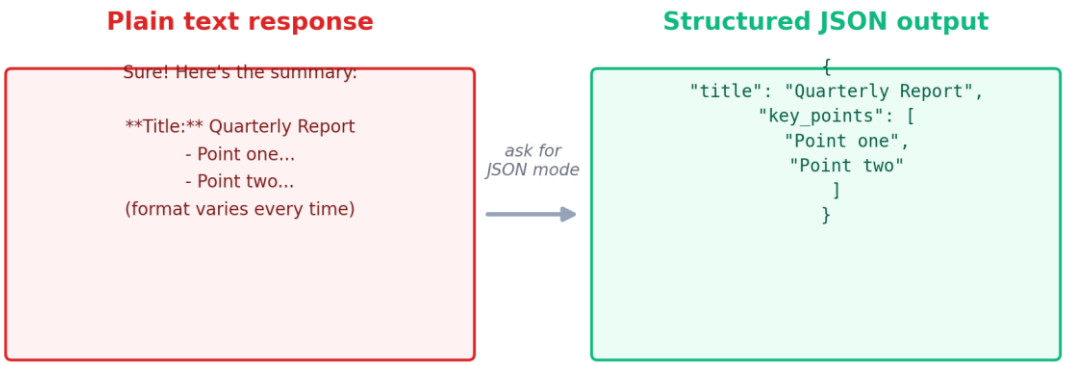

Figure 2.3 — Same request, dramatically easier to parse.

Requesting JSON mode tells the model to reply with valid JSON instead of prose:

```
import json

system_msg = (
    "Extract structured data. Reply with JSON only."
)
user_msg = (
    "Summarize: The Q3 report shows revenue "
    "up 12%, costs down 5%."
)

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": system_msg},
        {"role": "user", "content": user_msg}
    ],
    response_format={"type": "json_object"}
)
data = json.loads(response.choices[0].message.content)
print(data["summary"])


# example — plain text vs structured JSON
PLAIN TEXT  "Revenue increased by 12%, while costs decreased by 5%."
JSON        {"revenue_change_pct": 12, "cost_change_pct": -5}
Your code can now use data["revenue_change_pct"] directly.
```

Gemini version:

```python
import json
import os
from dotenv import load_dotenv
from google import genai
from google.genai import types

load_dotenv()

# Initialize client using your explicit api_key syntax
client = genai.Client(
    api_key=os.getenv("GEMINI_API_KEY")
)

system_msg = "Extract structured data. Reply with JSON only."
user_msg = "Summarize: The Q3 report shows revenue up 12%, costs down 5%."

# Generate structured JSON output using Gemini 3.6 Flash
response = client.models.generate_content(
    model="gemini-3.6-flash",
    contents=user_msg,
    config=types.GenerateContentConfig(
        system_instruction=system_msg,
        response_mime_type="application/json"
    )
)

data = json.loads(response.text)
print(data)
```

Function calling (sometimes called "tools") goes a step further: instead of asking the model to describe what it wants to do, you give it a list of real functions it can call, with their expected arguments. The model decides when to use one and returns which function to call with what arguments — your code still has to actually run it.

```python
import os
from dotenv import load_dotenv
from google import genai
from google.genai import types

load_dotenv()
client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))

def multiply(a: float, b: float) -> float:
    """Multiplies two numbers together."""
```

```
    return a * b

# Force Gemini to return a tool call instead of text
chat = client.chats.create(
    model="gemini-3.1-flash-lite",
    config=types.GenerateContentConfig(
        tools=[multiply],
        tool_config=types.ToolConfig(
            function_calling_config=types.FunctionCallingConfig(
                mode="ANY"  # Forces the model to generate a function call
            )
        )
    )
)

response = chat.send_message("What is 123 multiplied by 456?")

# With mode="ANY", response.function_calls is guaranteed to exist
tool_call = response.function_calls[0]
print("Model requested tool:", tool_call.name)
print("Arguments extracted:", tool_call.args)

# Execute local Python function
result = multiply(**tool_call.args)
print("Execution result:", result)
```

expected output:

```
Model requested tool: multiply
Arguments extracted: {'b': 456, 'a': 123}
Execution result: 56088
```

example — function calling: model chooses, your code executes

1. User asks: "What is 123 multiplied by 456?"
2. Model returns: multiply(a=123, b=456)
3. Your application actually runs multiply(123, 456).
4. Your code sends the real result (56088) back to the model.
5. Model turns that result into a natural final answer.

```
import os
from dotenv import load_dotenv
from google import genai
from google.genai import types
```

```python
load_dotenv()
client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))

# 1. Define multiple local tools
def add(a: float, b: float) -> float:
    """Adds two numbers together."""
    return a + b

def subtract(a: float, b: float) -> float:
    """Subtracts b from a."""
    return a - b

def multiply(a: float, b: float) -> float:
    """Multiplies two numbers together."""
    return a * b

def divide(a: float, b: float) -> float:
    """Divides a by b."""
    return a / b if b != 0 else float("nan")

# Map function names to Python functions for dynamic execution
available_tools = {
    "add": add,
    "subtract": subtract,
    "multiply": multiply,
    "divide": divide,
}

# 2. Register all tools in the chat configuration
chat = client.chats.create(
    model="gemini-3.5-flash-lite",
    config=types.GenerateContentConfig(
        tools=[add, subtract, multiply, divide],
        tool_config=types.ToolConfig(
            function_calling_config=types.FunctionCallingConfig(
                mode="ANY"  # Forces the model to pick a tool call
            )
        )
    )
)

# Step 1: Send a prompt requiring a specific tool
response = chat.send_message("What is 100 divided by 4?")

# Step 2: Extract requested tool call
tool_call = response.function_calls[0]
tool_name = tool_call.name
```

```
tool_args = tool_call.args

print("Model requested tool:", tool_name)
print("Arguments extracted:", tool_args)

# Step 3: Dynamically execute the requested function
if tool_name in available_tools:
    result = available_tools[tool_name](**tool_args)
    print("Execution result:", result)
```

Important: the model requests the action; it does **not** execute your Python function by itself.

The model never actually performs the multiplication calculation itself — it recognizes that multiply is the right tool and tells you which arguments to call it with. Your code runs the real function, and typically sends the result back to the model in a follow-up message so it can phrase a final answer.

PRO TIP *Reach for JSON mode when you just need clean, parseable data back. Reach for function calling when the model needs to trigger an actual action in your system — a database lookup, a real API call, a calculation your code (not the model) should perform.*

✓ QUICK CHECK — Structured Outputs & Function Calling

Q1. What problem does JSON mode solve?

Answer: *It avoids unpredictable, hard-to-parse free-form text by having the model reply with valid, structured JSON instead.*

Q2. In function calling, who actually executes the function — the model or your code?

Answer: *Your code. The model only decides which function to call and with what arguments; it doesn't execute anything itself.*

04 Awareness Briefing: What's Next

Three more ideas come up constantly once you start building real AI applications. You won't write code for these today — the goal is just recognizing them by name and having a rough mental model of what each one is for, so they aren't a mystery the next time you encounter them.

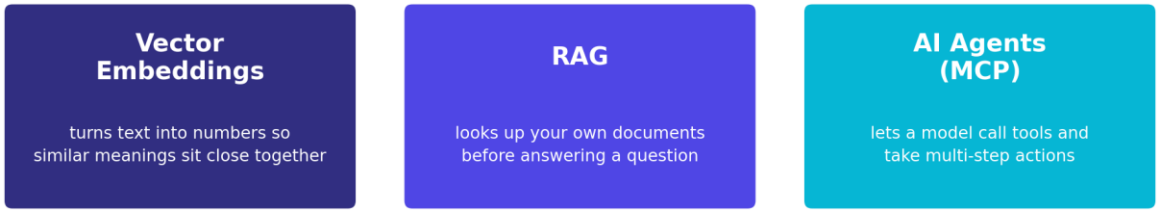

Figure 2.4 — Three ideas worth recognizing, even before you build with them.

Vector **embeddings** turn a piece of text into a long list of numbers in a way where texts with similar meaning end up numerically close together. This is the foundation that lets a computer search "by meaning" instead of by exact keyword match.

```
# example — semantic search with embeddings
Document A  "How to repair a car engine"
Query       "fixing an automobile motor"
Keyword search may miss it because the exact words differ.
Embedding search can match them because their meanings are similar.
```

RAG (Retrieval-Augmented Generation) uses embeddings to look up relevant chunks of your own documents — a knowledge base, a set of PDFs, your company's internal wiki — and feeds them into the prompt before the model answers. It's how a chatbot can accurately answer questions about content the model was never trained on.

```
# example — RAG with a company policy document
User asks  "How many annual-leave days do employees get?"
Retrieve   Search the company handbook for the relevant leave-policy chunk.
Augment    Put that chunk into the model context.
Generate   Answer using the retrieved policy instead of guessing from training memory.
RAG = Retrieve -> Add context -> Generate.
```

AI Agents, sometimes built using a standard called MCP (Model Context Protocol), let a model decide to call external tools and take multiple steps toward an answer, rather than just replying once from what it already knows — the same idea from Day 1 of the prompt engineering course, now from the builder's side instead of the user's side.

```
# example — an AI agent as a multi-step loop
Goal   "Find tomorrow's meeting time and draft a reminder."
Step 1   Agent checks a calendar tool.
Step 2   It reads the returned meeting details.
Step 3   It may call another tool or compose the reminder.
Agent = model + tools + repeated decisions toward a goal.
```

PRO TIP *Don't feel behind for not building these yet — recognizing the vocabulary today means the next course that covers them in depth will make immediate sense instead of starting from zero.*

That's a wrap for the lecture portion of this course. In the lab, you'll put everything from both days together and build a small, complete AI application of your own.

✓ QUICK CHECK — Awareness Briefing: What's Next

Q1. What do vector embeddings let you search by, that keyword search can't?

Answer: *By meaning — texts with similar meaning end up numerically close together, even if they don't share exact keywords.*

Q2. What problem does RAG solve?

Answer: *It lets a model accurately answer questions about content it was never trained on, by looking up and including relevant documents before answering.*

☆ Check Your Understanding

Work through these on your own before checking the marked answer.

Practice Question 1

What does the 'system' role in a chat API message typically set?

A. The exact words the model must reply with

B. The model's overall behavior and personality for the whole conversation

C. The user's name

D. The API key

Practice Question 2

Why does a chatbot need to resend the whole conversation history on every API call?

A. It doesn't -- the API remembers previous calls automatically

B. The API has no memory of its own; each request is independent, so history must be resent to be 'remembered'

C. Only to satisfy a rate limit

D. Only when using function calling

Practice Question 3

What will this code do?

```
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[...],
    response_format={"type": "json_object"}
)
```

A. Request the model reply with valid JSON instead of free-form text

B. Make the API call run twice as fast

C. Automatically call a function in your code

D. Disable the system role

Practice Question 4

In function calling, when the model returns a tool call, what has actually happened?

A. The function has already run and returned a result

B. The model has decided which function to call and with what arguments -- your code still has to run it

C. The API key has been validated

D. Nothing -- function calling requires no code on your end

Practice Question 5

Which of these is the best one-line description of RAG?

A. A way to make API calls faster

B. Looking up your own documents and feeding relevant pieces into the prompt before the model answers

C. A type of vision-only model

D. A JSON formatting option

Answers

Practice Question 1

What does the 'system' role in a chat API message typically set?

A. The exact words the model must reply with

B. The model's overall behavior and personality for the whole conversation ✓

C. The user's name

D. The API key

Why: *The system message sets overall instructions and behavior that apply for the whole conversation, distinct from the specific back-and-forth in user/assistant messages.*

Practice Question 2

Why does a chatbot need to resend the whole conversation history on every API call?

A. It doesn't -- the API remembers previous calls automatically

B. The API has no memory of its own; each request is independent, so history must be resent to be 'remembered' ✓

C. Only to satisfy a rate limit

D. Only when using function calling

Why: *Every request to the API is stateless — if you want the model to have context from earlier turns, you must include that history yourself in the messages list every time.*

Practice Question 3

What will this code do?

```
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[...],
    response_format={"type": "json_object"}
)
```

A. Request the model reply with valid JSON instead of free-form text ✓

B. Make the API call run twice as fast

C. Automatically call a function in your code

D. Disable the system role

Why: *response_format={"type": "json_object"} is how you request JSON mode, so the reply is structured, parseable JSON rather than unpredictable prose.*

Practice Question 4

In function calling, when the model returns a tool call, what has actually happened?

A. The function has already run and returned a result

B. The model has decided which function to call and with what arguments -- your code still has to run it ✓

C. The API key has been validated

D. Nothing -- function calling requires no code on your end

Why: *The model never executes anything itself. It identifies the right function and arguments; your application code is responsible for actually running that function.*

Practice Question 5

Which of these is the best one-line description of RAG?

A. A way to make API calls faster

B. Looking up your own documents and feeding relevant pieces into the prompt before the model answers ✓

C. A type of vision-only model

D. A JSON formatting option

Why: *RAG (Retrieval-Augmented Generation) retrieves relevant content — often via embeddings — and includes it in the prompt, letting the model answer accurately about material it wasn't trained on.*